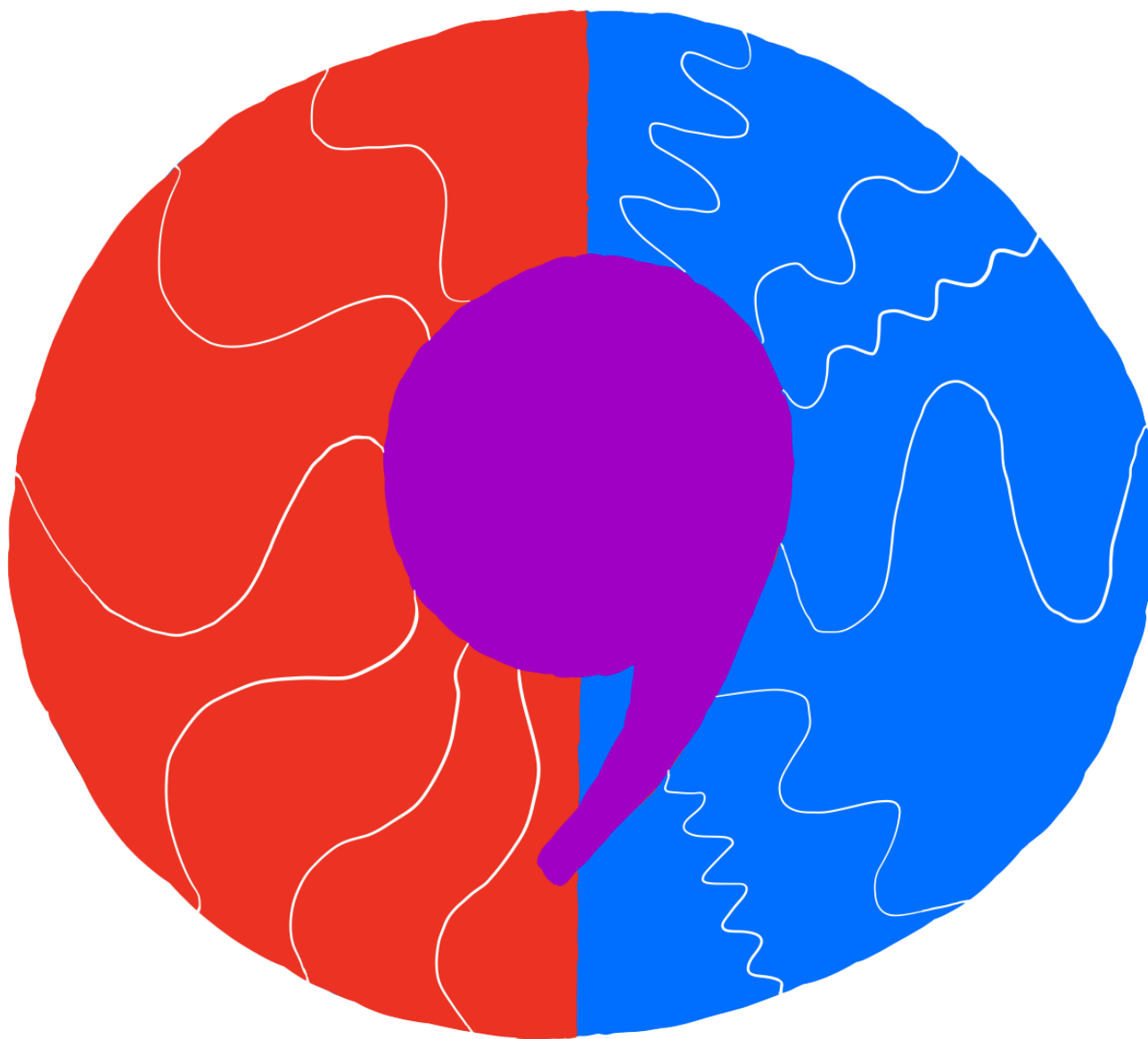


bind_tcp_agent: исходящий TCP из Mythic

 purpleshift.io/ru/articles/2026-08-14-bindtcpagent

12 августа 2026 г.



1. [Статьи/](#)

1. Что это такое#

[bind_tcp_agent](#) — это агент для [Mythic C2](#), который выступает в роли **P2P-моста** между Mythic и удалёнными агентами (Poseidon, Apollo), уже слушающими [TCP](#)-порты. В отличие от традиционных C2-агентов, этот агент **не генерирует нагрузку** — это «виртуальный колбэк», который динамически связывается с существующими агентами через TCP и ретранслирует C2-трафик.

Проще говоря: вы устанавливаете его внутри Mythic, используете команду `link` для подключения к Poseidon или Apollo, слушающему TCP-порт, и он становится

прозрачным ретранслятором для команд, передачи файлов, интерактивных шеллов, обратных пробросов портов и SOCKS-проксирования — всё через одно исходящее TCP-соединение.

Ключевые характеристики#

- **Только исходящие соединения:** `bind_tcp_agent` подключается к агентам, а не наоборот.
- **Поддержка P2P:** работает с многошаговыми цепочками: Mythic → `bind_tcp_agent` → Poseidon A → Poseidon B.
- **Поддержка SOCKS/HTTP-прокси:** соединения могут маршрутизироваться через SOCKS4, SOCKS5 или HTTP-прокси.
- **AES-256-CBC + HMAC-SHA256:** коммуникация может быть в открытом виде, зашифрована через AESPSK или EKE (обмен ключами RSA → сессионный ключ).

2. Проблема, которую решает `bind_tcp_agent`#

У Mythic C2 есть две категории C2-профилей: **egress** (агенты, которые сами выходят на связь с Mythic, например по HTTP, DNS, SMB) и **P2P** (агенты, которые маршрутизируются через другие агенты, например по TCP, SMB). И Poseidon, и Apollo поддерживают TCP в качестве P2P-профиля — они слушают TCP-порт и ждут, пока родительский агент подключится и ретранслирует задания.

Проблема в том, что **в самом Mythic нет встроенного egress-профиля, говорящего на протоколе TCP P2P**. Если вы разворачиваете Poseidon с TCP P2P-профилем внутри целевой сети, Mythic не имеет способа до него достучаться: Poseidon ждёт *входящего* TCP-соединения, но со стороны Mythic нет ничего, что могло бы его инициировать.

Агент `bind_tcp_agent` решает эту проблему, выступая в роли недостающего звена:

Mythic ↔ <code>bind_tcp_agent</code> (Docker)	←TCP→ Poseidon (target)	←TCP→ Poseidon (target)
(egress)	(P2P-мост)	(P2P-слушатель)
		(P2P-слушатель)

Агент живёт внутри контейнерной сети Mythic, нативно говорит на Mythic RPC и инициирует исходящие TCP-соединения к удалённым агентам, где бы они ни находились.

Сценарий: внутренний пентест с jump-хостом, исходящее соединение от mythic-сервера по tcp с помощью socks, http.

3. Команда `link#`

Команда `link` — сердце агента. Вот полная последовательность от команды оператора до активного опроса:

Фаза 0 — Создание виртуального колбэка#

Когда оператор собирает нагрузку `bind_tcp_agent` в Mythic:

1. `builder.py` создаёт виртуальный колбэк через `SendMythicRPCCallbackCreate()`
2. Регистрируется egress-ребро (`self → self`), чтобы Mythic воспринимал его как маршрутизируемый колбэк
3. `builder.py` возвращает `b""` — нагрузка не генерируется.

Фаза 1 — Команда `link#`

```
[Ⓢ 11-Jun-2026 14:05]-[👤 os]-[# T-83])
└─> link 127.0.0.1:18888
Connected to 127.0.0.1:18888, waiting for agent checkin...Successfully linked to 127.0.0.1:18888
Agent checked in successfully
New callback UUID: f0a92f09-a8a5-42d1-b67e-9edf001c7008
P2P edge created: C-11 -> new callback
```

1. `link.py` парсит аргументы (IP, порт, опциональные настройки прокси)
2. Вызывает `BindtcpRPC.Connect()` для установки TCP-сокета.

Фаза 2 — TCP-соединение#

Класс `TcpConnection` (`connection.py`) управляет соединениями:

- Прямое TCP или SOCKS4/5/HTTP-прокси (через `PySocks`)
- `TCP_NODELAY`, `SO_KEEPALIVE` (`KEEPIDLE=30с`, `KEEPINTVL=10с`, `KEEPCNT=3`)
- Таймаут на подключение 30 секунд
- `_recv_exact()` с дедлайном для устойчивости к частичным чтениям

Фаза 3 — Регистрация агента#

После установки соединения `ReadAndForwardCheckin()` (`checkin.py`) читает первое сообщение агента:

1. Получает сырые байты из TCP-сокета
2. Декодирует сообщение из Base64
3. Извлекает **UUID** (первые 36 байт) — идентифицирует агента

4. Определяет режим шифрования, пытаясь распарсить тело:

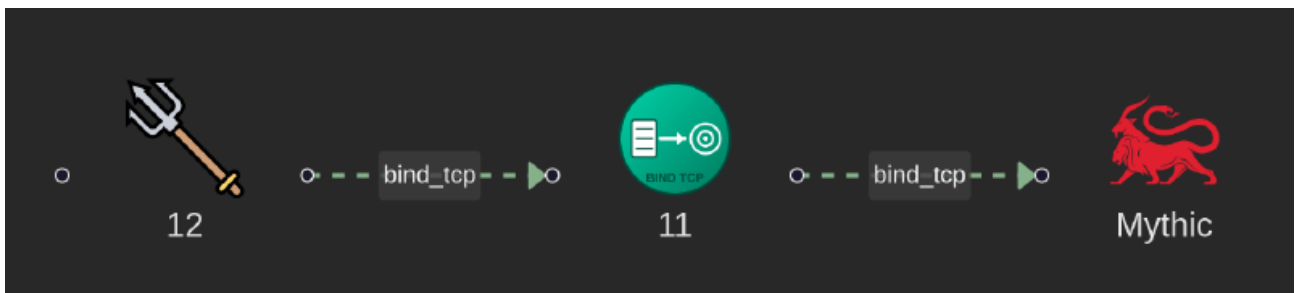
- Проверяет `json.loads()` → если сработало, сообщение **в открытом виде**
- Проверяет AES-256-CBC расшифровку с сессионным ключом → если сработало, режим **EKE**
- Проверяет AES-256-CBC расшифровку с AESPSK → если сработало, режим **AESPSK**

5. Маршрутизирует по action:

- "checkin" → Создает колбэк в Mythic, регистрирует P2P-ребро, запускает опрос
- "staging_rsa" → Обмен ключами RSA (EKE), затем рекурсия для зашифрованного чекина
- "post_response" / "get_tasking" → Агент переподключается с существующим колбэком

Фаза 4 — Регистрация P2P-ребра#

После успешного чекина P2P-ребро `bind_tcp_agent` → `remote_agent` регистрируется в Mythic, включая маршрутизацию команд и трафика через граф.



Фаза 5 — Фоновый опрос#

Для соединения запускается опрос, работающий на протяжении всей жизни агента.

4. Режимы шифрования#

`bind_tcp_agent` поддерживает три режима шифрования, соответствующие возможностям Poseidon.

4.1 Открытый текст (Plaintext)#

Сообщение агента — чистый JSON. Используется для тестирования или когда шифрование обеспечивается на более высоком уровне.

```
Wire: base64( UUID(36) + JSON(body) )
```

4.2 AESPSK (Pre-Shared Key)#

Каждое сообщение шифруется AES-256-CBC + HMAC-SHA256.

Wire: base64(UUID(36) + IV(16) + ciphertext + HMAC(32))

Ключ запрашивается из Mythic через `GetPayloadEncryptionKey()`, если ещё не закеширован.

4.3 EKE (Encrypted Key Exchange)<#>

EKE использует RSA-OAEP для обмена временным AES-256 сессионным ключом. Поток полностью совпадает с Go-реализацией Poseidon:

1. Агент отправляет `staging_rsa` со своим RSA-публичным ключом (PEM-кодированный, формат PKCS#1 или SPKI)
2. `bind_tcp_agent` генерирует случайный 32-байтный сессионный ключ
3. Сессионный ключ шифруется **RSA-OAEP (SHA-1)** и отправляется обратно
4. Агент расшифровывает своим приватным ключом
5. Все последующие сообщения используют AES-256-CBC + HMAC-SHA256 с сессионным ключом

5. Цикл опроса<#>

Каждое связанное соединение получает свой собственный фоновый поток опроса. Здесь происходит основная работа. Цикл повторяется до отключения или намеренного unlink:

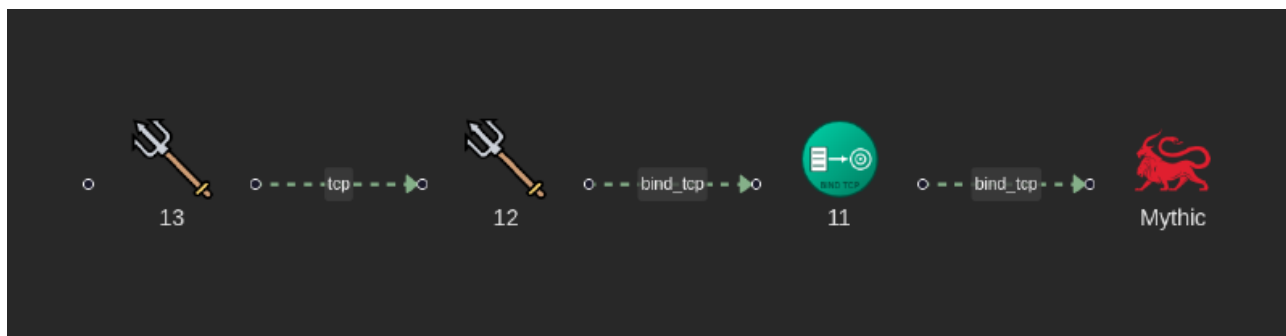
1. **Sleep** — ждёт `poll_interval ± jitter` секунд (пропускается при передаче файлов для пакетирования чанков)
2. **Получение заданий из Mythic** — вызывает `GetTasksFromMythic()` для получения ожидающих заданий и делегатов
3. **Сборка сообщения** — объединяет задания Mythic + очередь ответов делегатов + ожидающие данные `grfwd` в одно JSON-сообщение, шифрует при необходимости
4. **Отправка агенту** — записывает через TCP с префиксом UUID колбэка
5. **Чтение ответов** — цикл чтения ответов агента

6. P2P — многошаговый C2<#>

`bind_tcp_agent` поддерживает произвольную цепочку через промежуточные агенты. Делегат — это сообщение, предназначенное для дочернего агента, упакованное внутри C2-сообщения родителя.

Исходящий (Mythic → конечный агент)<#>

Mythic → `bind_tcp_agent` → Агент A → Агент B



7. Обработка переподключений#

Когда поток опроса сталкивается с ошибкой чтения или отправки, он входит в цикл переподключения:

1. Экспоненциальная задержка: 2с, 4с, 8с, 16с, 32с
2. Максимум 5 попыток, затем попытки переподключений останавливаются
3. Закрывает старый сокет → Переподключает TCP → Читает чекин
4. Если тот же UUID колбэка → Возобновляет опрос
5. Если **НОВЫЙ** UUID колбэка (свежий EKE) → Старый поток завершается (новый поток опроса уже запущен в обработчике чекина)

Персистентность состояния через API agentstorage Mythic (save_to_storage / load_from_storage) гарантирует, что ключи, маппинги колбэков и очереди делегатов переживут перезапуск контейнера:

Состояние восстанавливается при старте:

- Ключи пейлоада (AESPSK)
- Сессионные ключи (EKE)
- Маппинги маршрутизации колбэков
- Временные маппинги UUID (для незавершённого EKE)
- Очереди ответов делегатов
- Метаданные соединений (для relink)

8. Протокол передачи#

TCP-фрейминг#

Сообщения разбиваются на куски по 30 КБ с 12-байтным заголовком:

SIZE (4B)	TOTAL_CHUNKS	CHUNK_NUM	CHUNK_DATA
big-end	(4B)	(4B)	(size - 8 bytes)

Получатель собирает чанки по номеру и переупорядочивает.

Формат сообщения (после Base64-декодирования)<#>

UUID (36 байт)	IV (16B) [если зашифровано] + Ciphertext / Plaintext JSON body	HMAC-SHA256 (32B)
----------------	--	----------------------

Дерево решений шифрования<#>

Входящее сообщение

- └─ Пробуем `json.loads()` на теле → Успех? → Открытый текст, обрабатываем напрямую
- └─ Тело зашифровано:
 - └─ Сессионный ключ существует? → `aes_decrypt(session_key)`
 - └─ Ключ пейлоада закеширован? → `aes_decrypt(payload_key)`
 - └─ Ни первое, ни второе? → `GetPayloadEncryptionKey()` из Mythic RPC

R2P-обёртка делегатов<#>

Делегаты — это base64-кодированные сообщения, завёрнутые внутрь C2-сообщения родителя:

```
{
  "action": "get_tasking",
  "delegates": [
    {
      "uuid": "<connection_uuid>",
      "message": "<base64( child_uuid(36) + encrypted_body )>",
      "c2_profile": "tcp"
    }
  ]
}
```

9. Сборка и развёртывание<#>

Требования<#>

- Mythic C2 3.4+ (развёртывание на Docker)
- Контейнер Python 3.11 (собирается из исходников в Docker)

Сборка<#>

С сервера Mythic:

```
sudo /opt/Mythic/mythic-cli install folder -f /path/to/bind_tcp_agent
```

Это устанавливает тип пейлоада, C2-профиль и транслятор в контейнерную сеть Mythic.

Зависимости#

Все зависимости устанавливаются в многоступенчатой сборке Docker:

- mythic-container==0.6.9 — Python SDK Mythic
- pycryptodome — шифрование AES-256-CBC
- cryptography — RSA-OAEP, X.509, парсинг ASN.1
- PySocks — поддержка SOCKS4/5 и HTTP-прокси
- asn1crypto — парсинг PKCS#1 RSAPublicKey

Поддерживаемые агенты#

- **Poseidon** (Linux, macOS) — TCP-профиль
- **Apollo** (C#, Windows) — TCP-профиль

Использование#

Команда	Описание
link <ip> <port> [proxy_host proxy_port [proxy_type]]	Подключиться к удалённому агенту
sleep <callback_uuid> <interval> [jitter]	Установить интервал опроса
unlink <target_uuid>	Отключиться и очистить
relink <agent_callback_uuid>	Переподключиться из сохранённых метаданных

10. Заключение#

bind_tcp_agent закрывает конкретный пробел в экосистеме Mythic: он даёт возможность соединить egress-сеть Mythic с TCP-ориентированными P2P-агентами через исходящее соединение.

Проект имеет открытый исходный код и доступен для сообщества Mythic.

Приветствуются вклады и баг-репорты: https://github.com/olegsenko/bind_tcp_agent/

Related
